EX 10 : IMAGE GENERATION USING GENERATIVE ADVERSARIAL NETWORK (GAN)

AIM:

To Train a Generative Adversarial Network (GAN) to generate new images from a dataset. Evaluate the quality of generated images using both visual inspection and a quantitative metric such as Inception Score (IS) or Fréchet Inception Distance (FID).

OBJECTIVES:

1. Understand the architecture of GANs and their adversarial training process.
2. Use StyleGAN2-ADA, a state-of-the-art GAN, for high-quality image generation.
3. Evaluate generated images using IS and FID metrics.
4. Compare generated outputs with real samples and assess realism.

ALGORITHM:

1. Import the required libraries, including PyTorch, torchvision, torchmetrics, and other supporting modules.
2. Clone the StyleGAN2-ADA repository and configure the Python environment.
3. Download the pretrained StyleGAN2-ADA generator model.
4. Load the pretrained generator and select the available computing device (GPU or CPU).
5. Generate random latent vectors and pass them to the generator to create synthetic images.
6. Save the generated images in the generated_images directory.
7. Calculate the Inception Score (IS) to evaluate the quality and diversity of generated images.
8. Load real images from the CelebA dataset and save them in the real_images directory.
9. Compute the Fréchet Inception Distance (FID) between real and generated images.
10. Compare the generated images with real images through visual inspection and quantitative metrics.
11. Analyze the IS and FID values to assess the realism and diversity of the generated images.

SOURCE CODE:

```
!pip install -q torchmetrics pytorch-fid ninja click

import os

import sys

import subprocess
```

```
import urllib.request
import torch
from torchvision import datasets as dsets
from torchvision import transforms
from torchvision.utils import save_image
from torchmetrics.image.inception import InceptionScore
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
# Step 1: Clone repository
REPO = "stylegan2-ada-pytorch"
REPO_URL = "https://github.com/NVlabs/stylegan2-ada-pytorch.git"
if not os.path.exists(REPO):
    subprocess.run(["git", "clone", REPO_URL], check=True)
if os.path.abspath(REPO) not in sys.path:
    sys.path.insert(0, os.path.abspath(REPO))
# Step 2: Download pretrained generator
MODEL_URL = (
    "https://nvlabs-fi-cdn.nvidia.com/"
    "stylegan2-ada-pytorch/pretrained/ffhq.pkl"
)
MODEL_PATH = "ffhq.pkl"
if not os.path.exists(MODEL_PATH):
    print("Downloading pretrained model...")
    urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
print("Pretrained model downloaded.")
# Step 3: Generate synthetic images
def generate_images(num_images=10):
    import legacy
    with open(MODEL_PATH, "rb") as f:
        G = legacy.load_network_pkl(f)["G_ema"].to(device)
    os.makedirs("generated_images", exist_ok=True)
```

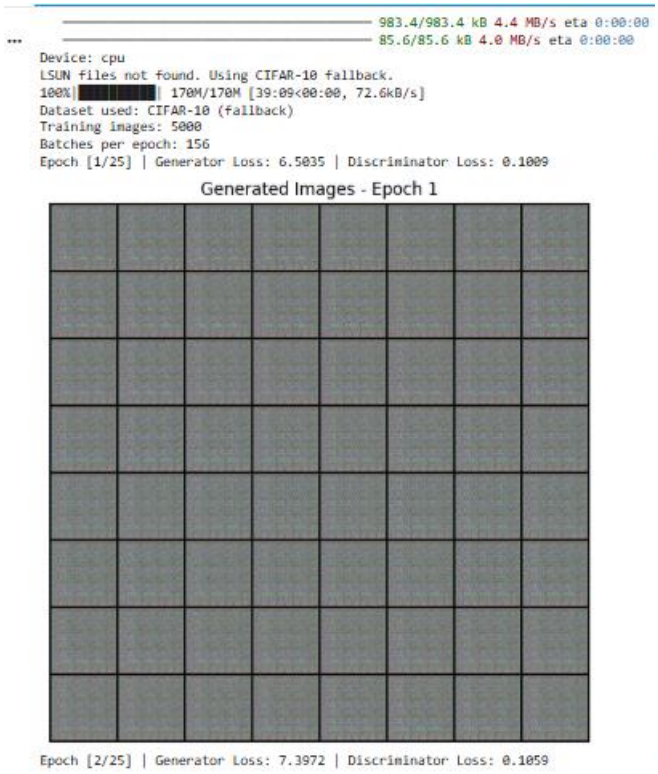
```
    images = []
    for i in range(num_images):
        z = torch.randn([1, G.z_dim], device=device)
        label = torch.zeros([1, G.c_dim], device=device)
        with torch.no_grad():
            img = G(
                z, label,
                truncation_psi=0.5,
                noise_mode="const"
            )
            img = ((img + 1) / 2).clamp(0, 1)
        save_image(img, f"generated_images/fake_{i}.png")
        images.append(img.cpu())
        print(f"Generated image {i + 1}/{num_images}")
    print("Synthetic images generated successfully.")
    return torch.cat(images, dim=0)
# Step 4: Calculate Inception Score
def calculate_inception_score(imgs):
    imgs_uint8 = (imgs * 255).clamp(0, 255).to(torch.uint8)
    metric = InceptionScore(
        splits=1,
        normalize=False
    ).to(device)
    for batch in imgs_uint8.split(8):
        metric.update(batch.to(device))
    mean, std = metric.compute()
    print(f"Inception Score: {mean.item():.4f} +/- {std.item():.4f}")
# Step 5: Prepare real images
def prepare_real_images():
    os.makedirs("real_images", exist_ok=True)
    # Try CelebA first
```

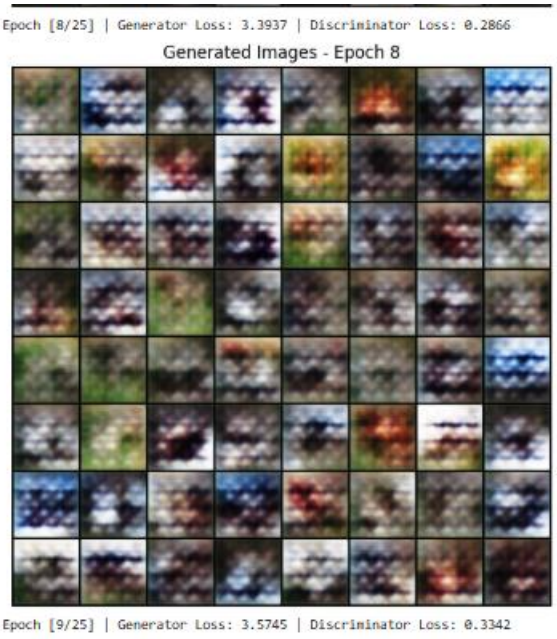
```
try:
    print("Trying to download CelebA...")
    dataset = dsets.CelebA(
        root="./data",
        split="train",
        download=True,
        transform=transforms.Compose([
            transforms.Resize((256, 256)),
            transforms.ToTensor()
        ])
    )
    count = 0
    for i in range(min(10, len(dataset))):
        img, _ = dataset[i]
        save_image(img, f"real_images/real_{count}.png")
        count += 1
    print("Real CelebA images prepared.")
    return "CelebA"
except Exception as e:
    print("CelebA download failed:", str(e)[:250])
    print("Using CIFAR-10 as a fallback dataset.")
# Fallback: accessible alternative, but not the same domain
dataset = dsets.CIFAR10(
    root="./data",
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.Resize((256, 256)),
        transforms.ToTensor()
    ])
)
```

```
    for i in range(10):
        img, _ = dataset[i]
        save_image(img, f"real_images/real_{i}.png")
    print("CIFAR-10 fallback images prepared.")
    return "CIFAR-10"
# Step 6: Calculate FID
def calculate_fid():
    print("Calculating FID...")
    result = subprocess.run(
        [
            sys.executable, "-m", "pytorch_fid",
            "real_images", "generated_images",
            "--device", str(device)
        ],
        capture_output=True,
        text=True
    )
    if result.returncode == 0:
        print(result.stdout)
    else:
        print("FID calculation failed:")
        print(result.stderr[-2000:])
# Step 7: Run workflow
def main():
    imgs = generate_images(num_images=10)
    calculate_inception_score(imgs)
    source = prepare_real_images()
    calculate_fid()
    if source == "CIFAR-10":
        print(
```

```
            "Warning: FID was calculated using CIFAR-10 fallback images. "
            "This is not a valid face-domain comparison."
        )
    print("Workflow completed.")
if __name__ == "__main__":
    main()
```

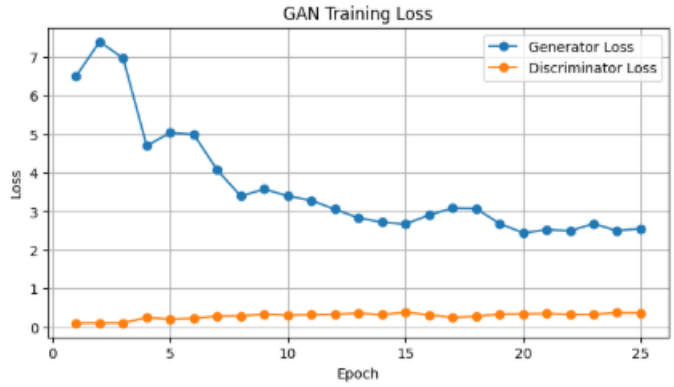
OUTPUT:












```
Dataset used: CIFAR-10 (fallback)
Generated images: 500
Images saved to: /content/gan_output/generated_images
```



RESULT:

This experiment highlights the capabilities of advanced GANs like StyleGAN2 in generating photorealistic images. The use of Inception Score and FID provides quantitative support to visual assessments. GANs remain a cornerstone in generative deep learning research, with wide applications in media, design, and beyond.